

Aflevering P2 - Mikkel Moth Billing

Vi fortsætter med at lade os inspirere af den japanske Sangaku tradition. Denne gang er det med et matematisk problem formuleret af 12-årige Imahori Yakichi i 1790, hvor han satte sin malede træplade op i Nagaoka Tenman templet i Kyoto.

Imahori's problem går ud på at finde sidelængden, k , på et kvadrat, således at når kvadratet placeres i origo og omgives af to gange 4 cirkler med radius hhv a og b , så rører disse cirkler præcis kvadratet og hinanden. Cirklerne med radius a er centreret på $y = \pm x$, mens cirklerne med radius b er centreret på $x = 0$ eller $y = 0$.

Opgave 1

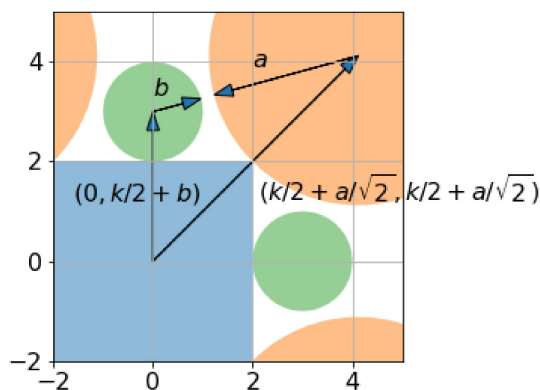
Gengiv den feedback du fik i forbindelse med Aflevering P1 og svar på evt. spørgsmål instruktoren har stillet.

Som feedback til aflevering P1 fik jeg at vide, at jeg ikke skulle bruge indexering i mine for loops, til at gå igennem lister. Istedet var der en metode til at gå igennem dem direkte, som jeg så har benyttet mig af i denne aflevering.

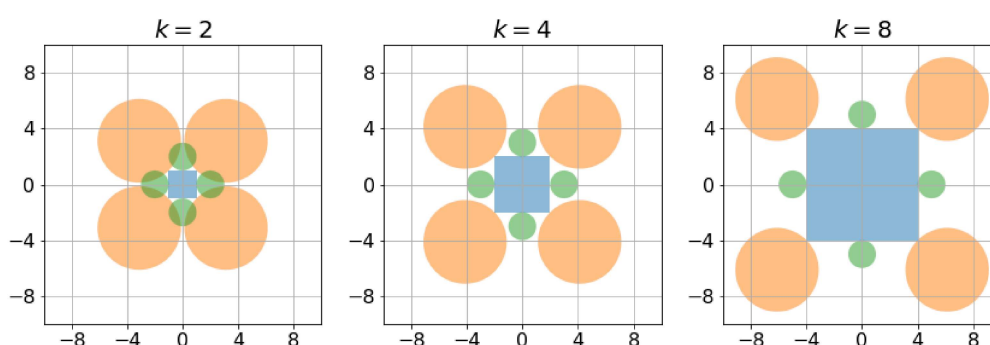
Opgave 2

Vi starter med at lave et plot for $a = 3$, $b = 1$ og forskellige k -værdier, der ikke nødvendigvis er de rigtige, hvorfor cirklerne ikke altid rører hinanden som de skal.

Du kan udlede det nødvendige om geometrien ud fra dette plot:



Skriv en (eller flere) funktion(er) så du på effektiv vis kan lave dette samlede plot for k med værdierne 2, 4 og 8. Med "effektiv vis" menes at du ikke gentager kode unødvendigt.

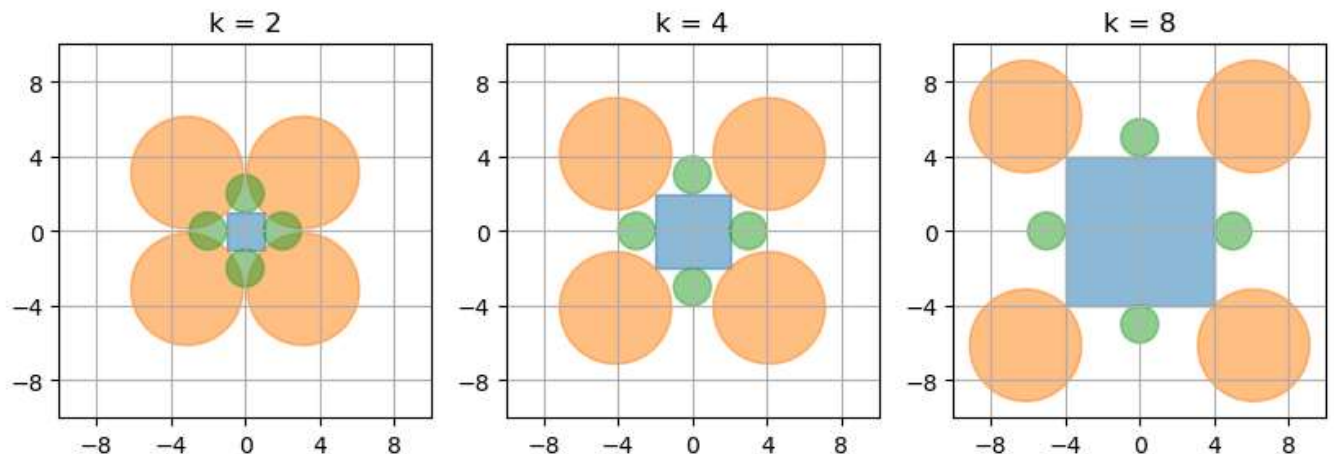


```
In [1]: import numpy as np
import matplotlib.pyplot as plt

fig, axes = plt.subplots(1,3, figsize=(10,8))
axes = axes.flatten()

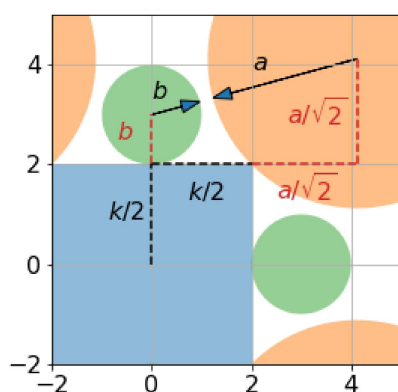
def plot(k, a, b, ax):
    #Firekant
    ax.fill_between([-k/2,k/2], [k/2,k/2], [-k/2,-k/2], color="tab:blue", alpha=0.5)
    #Orange cirkler
    theta = np.linspace(0, 2*np.pi, 1000, endpoint=False)
    x0_l = np.array([-k/2-a/np.sqrt(2), -k/2-a/np.sqrt(2), k/2+a/np.sqrt(2), k/2+a/np.sqrt(2)])
    y0_l = np.array([-k/2-a/np.sqrt(2), k/2+a/np.sqrt(2), -k/2-a/np.sqrt(2), k/2+a/np.sqrt(2)])
    for (x0,y0) in zip(x0_l,y0_l):
        x, y = a*np.cos(theta)+x0, a*np.sin(theta)+y0
        ax.fill(x,y, color="tab:orange", alpha=0.5)
    #Grønne cirkler
    x0_l = np.array([0,0,k/2+b,-k/2-b])
    y0_l = np.array([k/2+b,-k/2-b,0,0])
    for (x0,y0) in zip(x0_l,y0_l):
        x, y = b*np.cos(theta)+x0, b*np.sin(theta)+y0
        ax.fill(x,y, color="tab:green", alpha=0.5)

k_l = [2,4,8]
for (ax,k) in zip(axes,k_l):
    plot(k, 3, 1, ax)
    ax.set_title(f"k = {k}")
    ax.set_ylim(-10,10)
    ax.set_xlim(-10,10)
    ax.grid('on')
    ax.set_aspect('equal')
    ax.xaxis.set_major_locator(plt.MultipleLocator(4))
    ax.yaxis.set_major_locator(plt.MultipleLocator(4))
```



Opgave 3

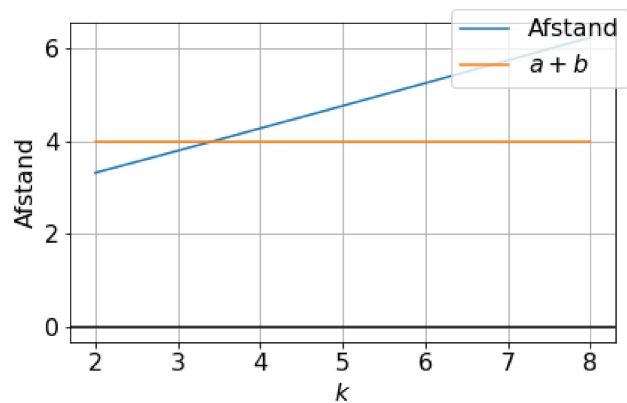
Kig på dette plot:



Læg mærke til at når k ikke har den rigtige værdi, så vil afstanden mellem to nabo-cirkler være "forkert", altså forskellig fra $a + b$.

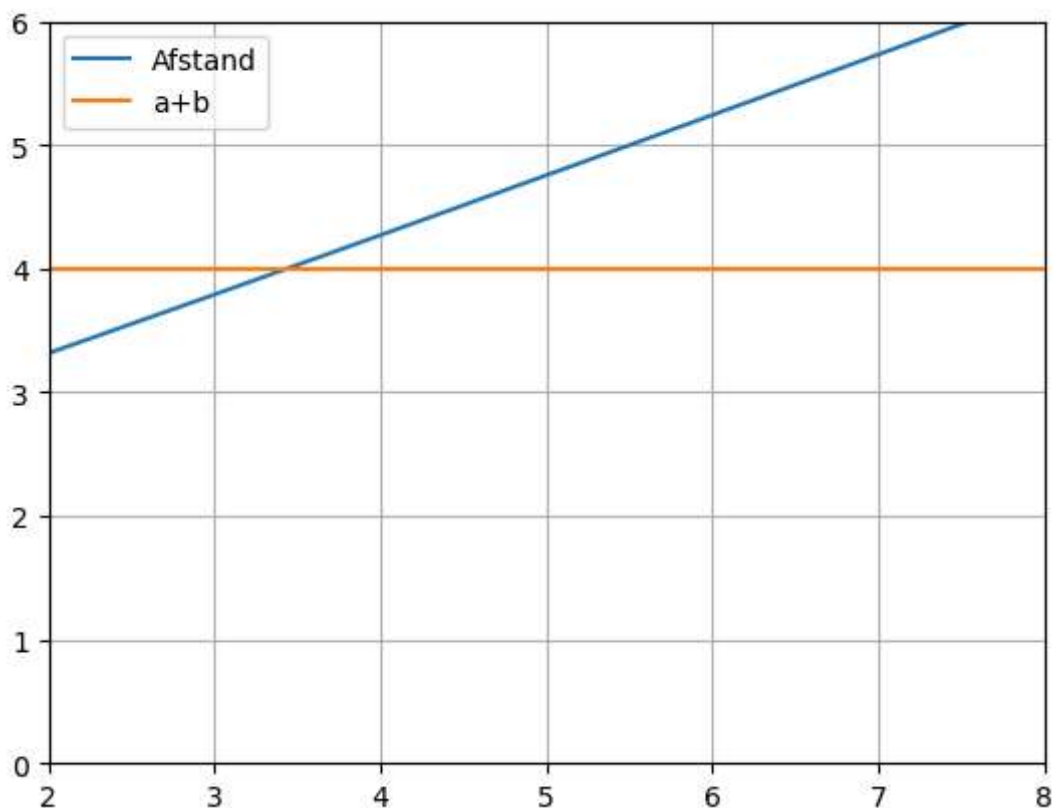
Skriv nu en funktion, der tager k som input og returnerer afstanden mellem centrene af to sådanne cirkler.

Plot funktionen for k i et rimeligt interval.

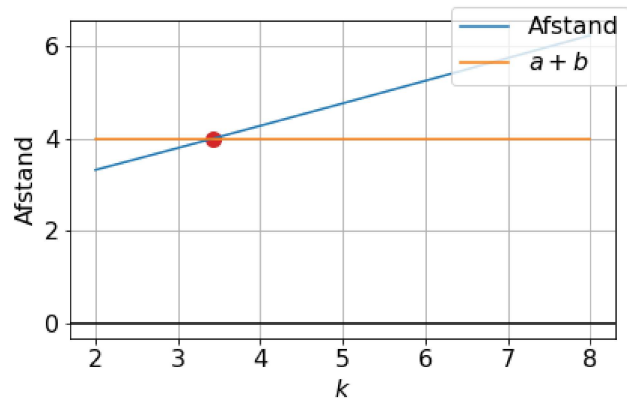


```
In [2]: def afstand(k):  
        return np.sqrt((k/2+a/np.sqrt(2))**2+(a/np.sqrt(2)-b)**2)  
  
k = np.arange(2,9)  
a=3  
b=1  
fig, ax = plt.subplots()  
ax.plot(k, afstand(k), label="Afstand")  
ax.axhline(y=a+b, color="tab:orange", label="a+b")  
ax.set_ylim(0,6)  
ax.set_xlim(2,8)  
ax.grid('on')  
ax.legend()
```

Out[2]: <matplotlib.legend.Legend at 0x23459228d10>

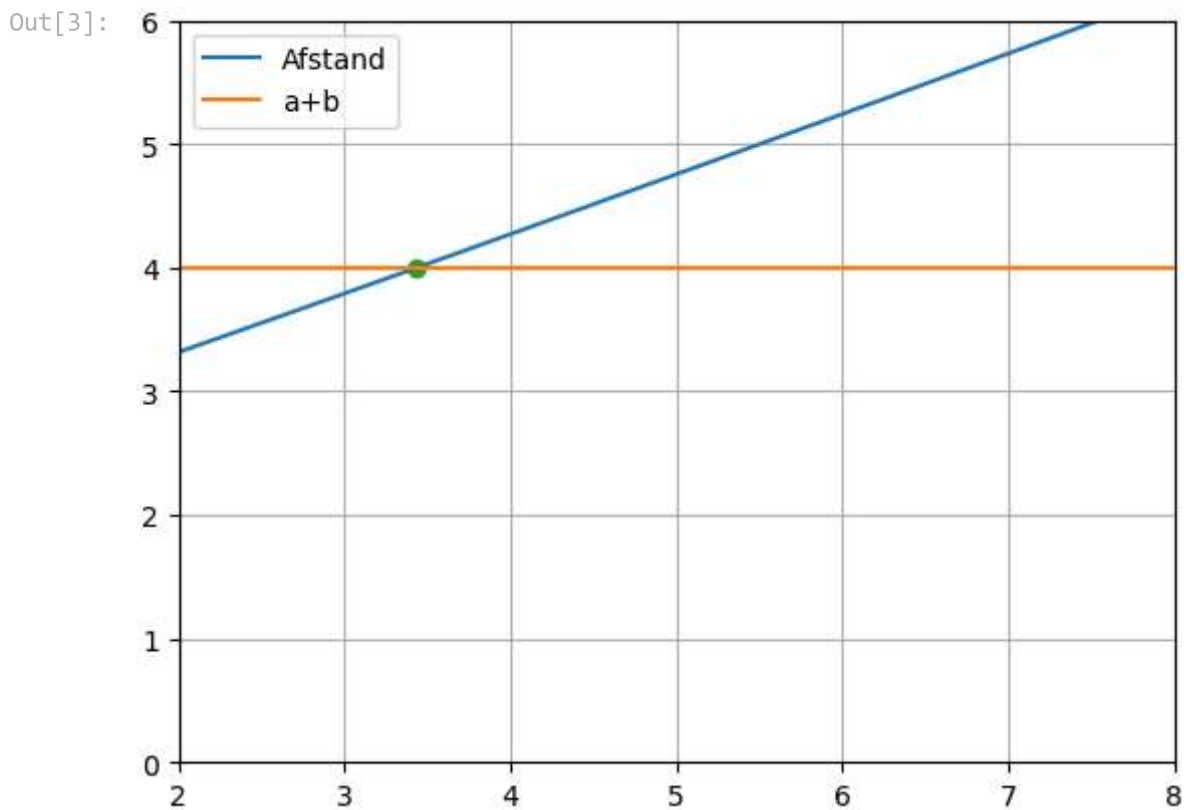


Find værdien af k for hvilken afstanden mellem ciklerne bliver præcis $a + b$. Benyt `fsolve` (eller `brentq`) fra `scipy.optimize`-biblioteket og indsæt skæringspunktet i afstands-plottet:



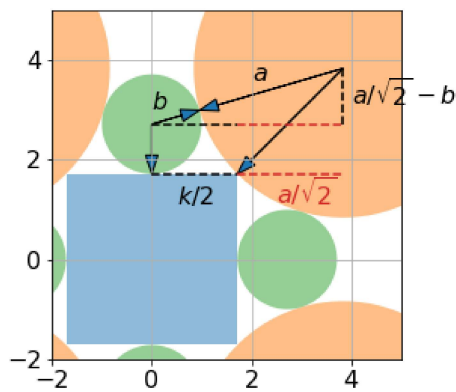
```
In [3]: from scipy.optimize import fsolve
p = fsolve(lambda k: afstand(k)-(a+b),3)
print(p)
ax.scatter(p, afstand(p), c="C2")
fig
```

[3.43658862]



Opgave 5

Se på denne tegning:



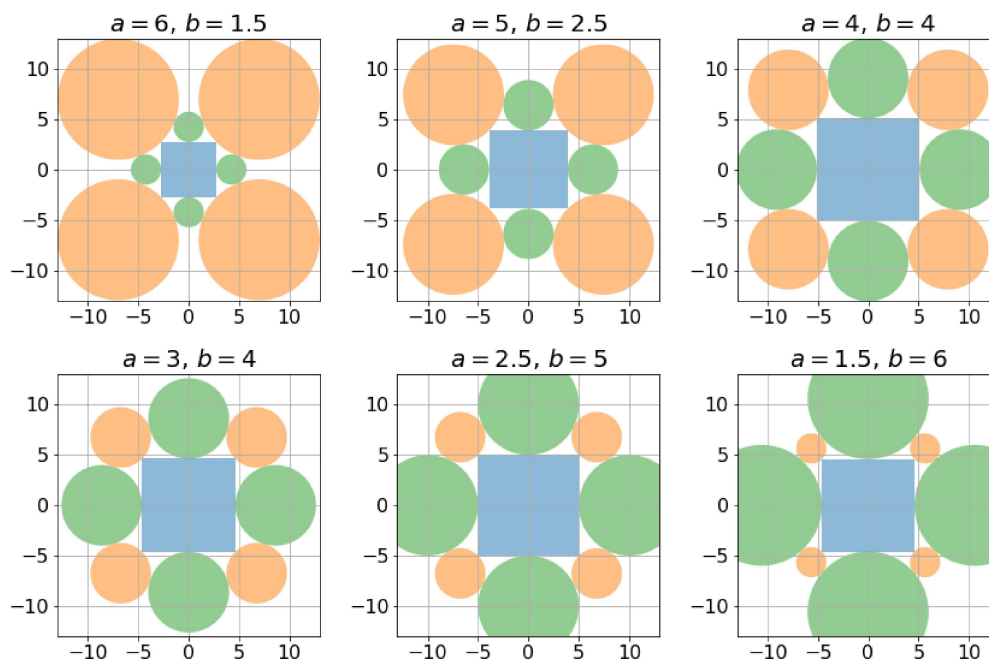
og opstil den matematiske betingelse for at nabo-cirkler netop rører hinanden. Gør det generelt hvor du endnu ikke har indsat værdier for a og b . Løs din ligning med SymPy. Du har nu et udtryk for det rette k som funktion af a og b . Konvertér denne funktion til en NumPy-funktion. Kald den med $a = 3$ og $b = 1$ og se at den giver det du fandt i opgave 4.

```
In [4]: import sympy as sp
k, a, b = sp.symbols("k,a,b")
f = sp.sqrt((k/2+a/sp.sqrt(2))**2+(a/sp.sqrt(2)-b)**2)-(a+b)
afstand_sp = sp.solve(f,k)
afstand_np = sp.lambdify([a,b], afstand_sp[1], "numpy")
afstand_np(3,1)
```

```
Out[4]: 3.436588619814917
```

Opgave 6

Lav nu dette plot, hvor værdierne for a og b varieres, mens der hele tiden benyttes den rigtige værdi for k :



Du skal bruge de funktioner du har udarbejdet i de tidligere opgaver, herunder NumPy-funktionen for $k(a,b)$.

```
In [6]: fig, axes = plt.subplots(2,3, figsize = (10, 8))
axes = axes.flatten()
a_l = [6,5,4,3,2.5,1.5]
b_l = [1.5, 2.5, 4, 4, 5, 6]
```

```

for (ax, a, b) in zip(axes, a_1, b_1):
    k = afstand_np(a,b)
    plot(k,a,b,ax)

    ax.set_title(f"a = {a}, b = {b}")
    ax.set_ylim(-12,12)
    ax.set_xlim(-12,12)
    ax.grid('on')
    ax.set_aspect('equal')
    ax.xaxis.set_major_locator(plt.MultipleLocator(5))
    ax.yaxis.set_major_locator(plt.MultipleLocator(5))

```

